

Basics of Bash Programming

Diego Garrido Martín

*UST, University of Vic, Vic
Computational Biology of RNA Processing, CRG, Barcelona*



- Basic concepts
- Basic commands
- Conditionals and loops
- Some tips
- Hands-on

I  Bash

1. Basic Concepts

The Bash shell

A screenshot of a terminal window with a black background. The prompt 'dgarrimar@5CG6255443:~\$' is displayed in green text, followed by a white cursor line.

```
dgarrimar@5CG6255443:~$
```

- The **Bash shell** is a command-line interpreter
- It can be accessed through a **terminal/console** (which may have a black, white or coloured background)
- The **prompt** is a string that tells us that the shell is ready
 - It may state the user and computer names and the current directory
 - It ends with \$, %, > or # (“\$” in the example above)
- A **command** is simply an instruction for the OS to perform a specific task
- After the prompt, you can type a command followed by the *Enter* key
- Forget about the mouse!

Bash commands

- How to run a command? `command options arguments`
 - When you execute a program or a simple command, you need to provide the **argument(s)**, i.e. the subject of the operation
e.g. `ls <directory name>`. `<directory name>` is the recipient of the action `ls`. It lists the contents of the directory.
 - **Parameters or options** modify the behavior of the command. They are preceded by “-” sign if they are single letter, or “--” if they have more than one letter
e.g. `ls -l <directory name>`. The `-l` option modifies the behavior of `ls`, which now prints the details of each file.

```
dgarrimar@5CG6255443:~$ ls -l software/
total 99708
drwxr-xr-x 1 dgarrimar dgarrimar      512 Aug  1 20:22 docker
-rw-rw-rw- 1 dgarrimar dgarrimar 102097042 Jul 19 21:06 rstudio-1.1.456-amd64.deb
dgarrimar@5CG6255443:~$
```

Bash commands

- The shell is **case-sensitive**, so “ls” and “LS” are not identical.
- You can use the “tab”  key to complete a path or the name of a file or command after typing the first characters.
 - If multiple files/commands share the same first characters, the shell will list all of them:

```
dgarrimar@5CG6255443:~$ ls file
file1  file2
dgarrimar@5CG6255443:~$ ls file
```

- The *up* and *down* arrows  and the *repag/avpag* keys help you to navigate the command history. The command `history` itself shows you all the commands that you have run.
- `man command` gives you detailed information on any command
e.g. `man ls`

Text editors and Bash scripts

- **Text editors.** There are several ways to write and edit files.
 - The most commonly used text editors are `vi`, `nano`, `emacs` and `gedit`.
 - You can choose the one you prefer, although during the course I will use `nano` due to its simplicity. You can open it by typing `nano` in your terminal.
- A **Bash script** is just a list of Bash commands within a file; it can be as simple as one command.
 - The symbol “`#`” can be used to introduce comments, that will not be interpreted by the shell.
 - It can start with a *shebang* `#!/bin/bash` which provides the instruction “*execute this script using the Bash shell*”, although you can omit it.
 - To distinguish them from other files, they may carry the extension `.sh`, although **file extensions are meaningless** to Bash.
 - To execute it, type `bash <script_name>`

Bash variables

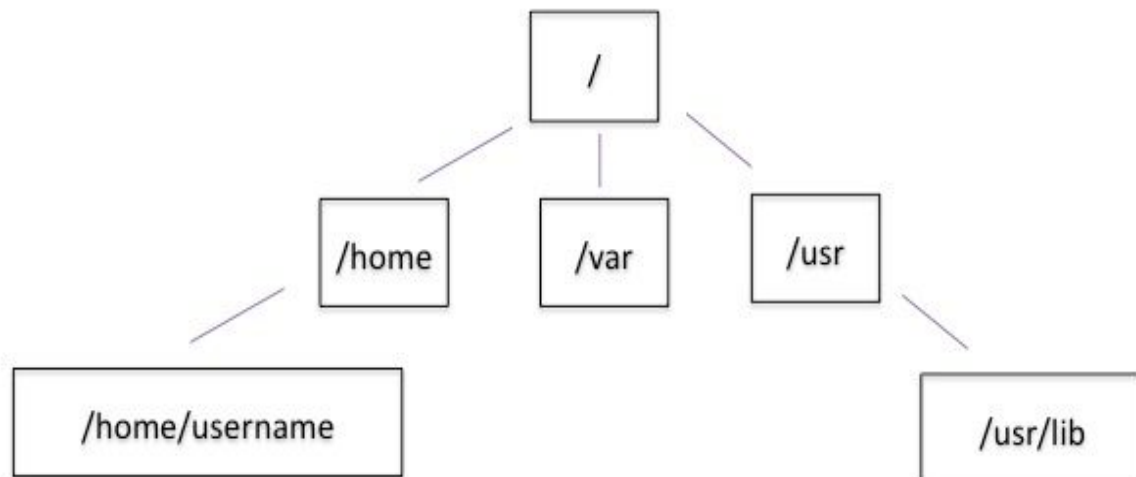
- **Environment variables.** pre-defined Bash variables for the current shell (they can also be set/modified by the user). You can see them by typing `env`. You may find useful some of them such as `PATH`, `HOME` or `PWD`.
- **User-defined variables**
 - You can define your own variables, using `=`.
e.g. `my_var="Hello World!"`
 - You can access their value using `$` e.g. print them `echo "$my_var"`
 - Note **there should not be spaces around `=`**.
 - Note you should use **plain text quotes** (`"`, `'` not `"`, `'`). They are the default in the terminal but not in some text editors (e.g. Microsoft Word).
 - **Double quotes allow interpolation, single quotes don't!**
e.g. `echo "$my_var"` prints `Hello World!`
`echo '$my_var'` prints `$my_var`

I/O redirection

- The output of your commands is by default printed in the terminal
- You can **redirect** the output of any command to a file using “>”,
i.e. `command > <file>` (e.g. `ls ~ > my_home.txt`)
 - This will create a new file if it does not exist, or **overwrite it** if exists.
 - use `2>` if you want to redirect the error of the command
 - use `&>` if you want to redirect both output and error
 - To append the output of your command to the end of an existing file, **without overwriting it**, use “>>”
 - Additionally, `command < <file>` : gives as input to your command the content of your file.
- Generally, the same file cannot be input and output:
`command <file> > <file>` will not work properly
- You can use the **pipe** “|” to connect several commands so that the output of the previous command is the input of the next one.
 - e.g. `ls -l ~ | head -n 2`
- You can combine all the previous! (we will see some examples later on)
e.g. `command4 <(command1 <file1> | command2 | command3) &> <file2>`

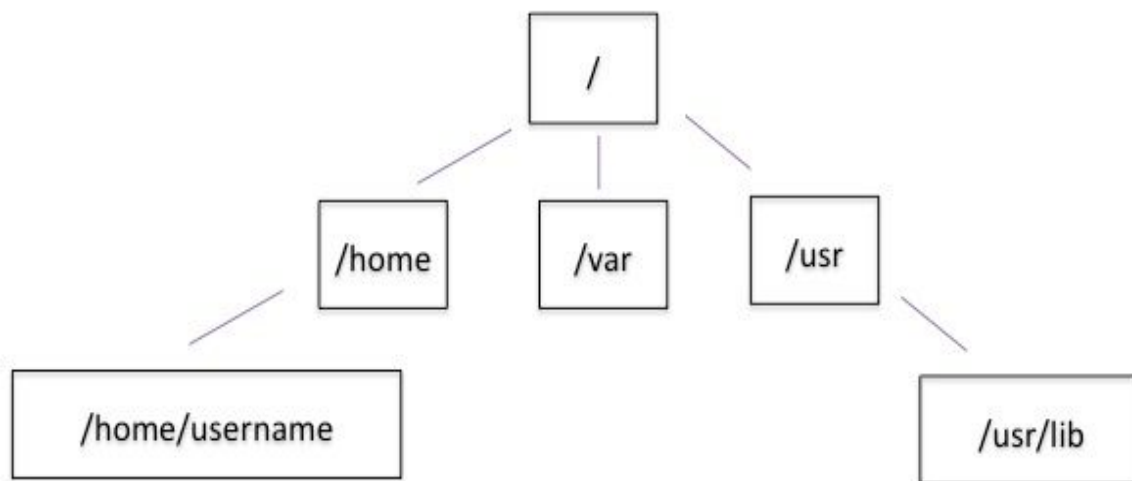
The Linux file system

The Linux **file system** is the collection of all **files**, **directories**, programs, etc. of the computer. It has a hierarchical structure similar to an upside-down tree.



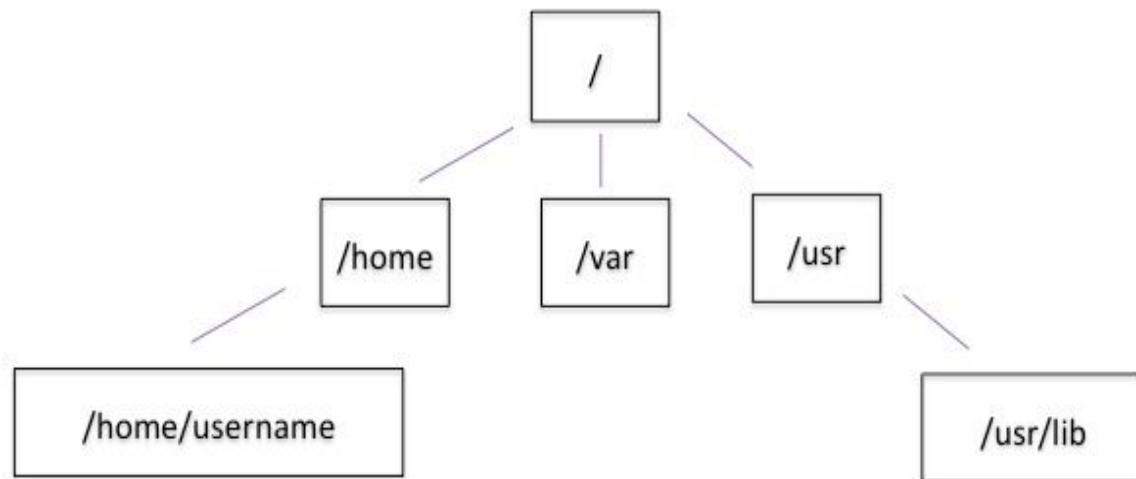
- The **root** directory (“/”) is the *mother* directory. There are several subdirectories, that may contain one or more directories and/or files.
- A **path** specifies the location of a file or directory in the file system: it indicates the directories to follow to reach that element, each separated by “/”. The path can be:
 - *absolute*: starts from the root. e.g. `/home/username/table.txt`
 - *relative*: starts from your current directory. e.g. `username/table.txt` if you are in `/home`

The Linux file system



- The **current directory** is where you are. It is indicated with “.”
 - you can omit it: `./Desktop/image.png` and `Desktop/image.png` are equivalent.
- The **parent directory** is indicated with “..”
 - If you are in `/home/username/Desktop` and want to reach `table.txt`, you can use the relative path `../table.txt`, instead of the longer absolute path `/home/username/table.txt`
- The **home directory** is the directory we typically work in. It is indicated with “~”, which is equivalent to `/home/username`. e.g. you could reach `table.txt` also using `~/table.txt`.

Navigate through the filesystem



- `pwd` prints your current directory
- `cd <dir>` moves to another directory
- `ls [-la] <dir>` lists the content of a directory [long format, also hidden files]
- `mkdir <dir>`, `rmdir <dir>` creates or remove an empty directory
- `touch <file>` creates an empty file
- `rm [-rf]` remove a file or directory (when removing directories use `-rf`)
- `mv <file> <dest>` moves file to dest (move to a folder, overwrite another file, rename)
- `cp [-r] <file> <dest>` copies file to dest (when copying directories use `-r`)
- `ln -s <file> <dest>` makes a symbolic link of file in dest

2. Basic commands

cat, head, tail, less, more, echo, wc

- `cat <file>` prints the content of a file or concatenates several files.
- `head [-n N] <file>; tail [-n N] <file>` prints the N first (last) lines of a file (default N=10)
- `less <file>; more <file>` shows the first part of a file (move with *space* and type *q* to quit)
- `echo <string|variable>` prints a string (or variable)
- `wc [-lwc]` reports the number of lines, words and characters of a file

```
dgarrimar@5CG6255443:~$ cat myfile
line 1
line 2
line 3
dgarrimar@5CG6255443:~$ head myfile -n 1
line 1
dgarrimar@5CG6255443:~$ echo "$HOME"
/home/dgarrimar
dgarrimar@5CG6255443:~$ echo "Hello World!"
Hello World!
dgarrimar@5CG6255443:~$ wc myfile
 3  6 21 myfile
dgarrimar@5CG6255443:~$ wc -l myfile
3 myfile
```

cut, paste

- `cut [-f N -d S --complement] <file>` prints the Nth column of the file (1-based). The default separator is `<tab>`, you can set a different one using `-d`. With `--complement` prints all the columns except the Nth.
- `paste <file1> <file2>` merges two files column-wise

```
dgarrimar@5CG6255443:~$ head myfile
11      12      13
21      22      23
31      32      33
dgarrimar@5CG6255443:~$ cut -f1-2 myfile
11      12
21      22
31      32
dgarrimar@5CG6255443:~$ cut -f2- myfile
12      13
22      23
32      33
dgarrimar@5CG6255443:~$ paste myfile myfile
11      12      13      11      12      13
21      22      23      21      22      23
31      32      33      31      32      33
```

join

- `join [-t] -1 <field1> -2 <field2> <file1> <file2>` joins `<file1>` and `<file2>` by the column of interest. Both files should be sorted by the fields of interest.
 - t defines the field separator (default is `<space>`).
 - Use `-t $'\t'` to set it to `<tab>`

See also `-j` options.

```
dgarrimar@5CG6255443:~$ cat lab
lab1    exp1
lab2    exp2
lab1    exp3
dgarrimar@5CG6255443:~$ cat sequence
exp1    rna
exp2    dna
exp3    proteins
dgarrimar@5CG6255443:~$ join -t $'\t' -1 2 -2 1 lab sequence
exp1    lab1    rna
exp2    lab2    dna
exp3    lab1    proteins
```


sort, uniq

- `sort [-kngr] <file>` prints the Nth column of the file (1-based)
 - `-k K,K` sort by column K (default 1)
 - `-n` sort integers (default alphanumeric)
 - `-g` sort floats
 - `-r` reverse order
- `uniq [-du] <file>` prints the entries of file (must be sorted) that are not duplicated (default), the duplicated (`-d`) or the unique (`-u`) ones.

```
dgarrimar@5CG6255443:~$ cut -f1 myfile | sort | uniq
a
b
c
d
e
dgarrimar@5CG6255443:~$ cut -f1 myfile | sort | uniq -d
a
```

```
dgarrimar@5CG6255443:~$ sort myfile
a      3.5
a      4
b     10
c      5
d      1
e      2
dgarrimar@5CG6255443:~$ sort -k2,2 myfile
d      1
b     10
e      2
a     3.5
a      4
c      5
dgarrimar@5CG6255443:~$ sort -k2,2nr myfile
b     10
c      5
a      4
a     3.5
e      2
d      1
```

diff

- `diff <file1> <file2>` compares the two files line by line.
outputs the differences, or nothing if files are identical
See also `man comm`

```
dgarrimar@5CG6255443:~$ cat myfile1 <(echo "=====") myfile2
dna
rna
proteins
=====
dna
rna
metabolites
dgarrimar@5CG6255443:~$ diff myfile1 myfile2
3c3
< proteins
---
> metabolites
```

“<” for lines present only in <file1>
“>” for lines present only in <file2>

- `grep [-vFfi] <pattern> <file>` prints the lines of file that match the pattern
 - v prints the lines that do **not** match the pattern
 - F looks only for fixed strings (makes grep faster!)
 - i ignores case (default case-sensitive)
 - f here `<pattern>` is a file containing a list of patterns

```
dgarrimar@5CG6255443:~$ cat myfile
Proteins
Environment
pRoteins
dgarrimar@5CG6255443:~$ grep [Pi]ro.* myfile
Proteins
EnviRnment
dgarrimar@5CG6255443:~$ grep -v teins myfile
Environment
dgarrimar@5CG6255443:~$ grep -i rot myfile
Proteins
pRoteins
```

Get more info about *regular expressions* [here](#). Remember also that some characters (" , | , etc.) should be **escaped** (\) to be matched literally)

gzip, gunzip, tar

- `gzip <file>; gunzip <file>` compresses/uncompresses a GNU zipped file.
See also `man bzip2`
- `tar` bundles a set of files and/or directories into a single archive or *tarball*
`tar -cf <archive_name.tar> <file1> [<file2>, ...]` to create a *tarball*
`tar -xf <archive.tar>` to extract files from a *tarball*
See also `tar -z` and `-v` options

```
dgarrimar@5CG6255443:~/tar_example$ mkdir dir2tar
dgarrimar@5CG6255443:~/tar_example$ touch dir2tar/myfile
dgarrimar@5CG6255443:~/tar_example$ ls; ls dir2tar/
dir2tar
myfile
dgarrimar@5CG6255443:~/tar_example$ tar -cf myarch.tar dir2tar/
dgarrimar@5CG6255443:~/tar_example$ ls
dir2tar  myarch.tar
dgarrimar@5CG6255443:~/tar_example$ rm -rf dir2tar/
dgarrimar@5CG6255443:~/tar_example$ ls
myarch.tar
dgarrimar@5CG6255443:~/tar_example$ tar -xf myarch.tar
dgarrimar@5CG6255443:~/tar_example$ ls; ls dir2tar/
dir2tar  myarch.tar
myfile
```

history, clear, env, wget, curl, chmod

- `history` shows the command history
- `clear` cleans the terminal (try also `CTRL+L`)
- `env` lists the environment variables and their content
- `wget <url>; curl <url>` download the file at `<url>`
- `chmod <file|dir>` allows to modify the permissions (reading/writing/executing) that a user or a group of users have over a file or directory. Often, you will need it to give execution permissions to your scripts:

```
dgarrimar@5CG6255443:~$ cat script.sh
#!/bin/bash

echo "Hello World!"
dgarrimar@5CG6255443:~$ bash script.sh
Hello World!
dgarrimar@5CG6255443:~$ ./script.sh
-bash: ./script.sh: Permission denied
dgarrimar@5CG6255443:~$
dgarrimar@5CG6255443:~$ chmod +x script.sh
dgarrimar@5CG6255443:~$
dgarrimar@5CG6255443:~$ ./script.sh
Hello World!
dgarrimar@5CG6255443:~$
```

- `sed s/pattern/subst/' <file>` substitutes the first occurrence of *pattern* in **every line** (default) of *file* for the string *subst*.
 - To substitute all occurrences use `'s/pattern/subst/g'`
 - You can use regular expressions as *patterns*.
 - You can interpolate variables (double quotes!)
 - You can also use `sed` to remove the initial line(s) of your file, e.g. `sed '1d' <file>`
 - You can use any element as separator, e.g. `'s[pattern[subst['`

```
dgarrimar@5CG6255443:~$ cat myfile
Hello World!
Hello Bioinformatics!
dgarrimar@5CG6255443:~$ sed 's/l/g/g' myfile
Heggo Worgd!
Heggo Bioinformatics!
dgarrimar@5CG6255443:~$ var="Ciao"; sed "s/^Hello /$var /" myfile
Ciao World!
Ciao Bioinformatics!
dgarrimar@5CG6255443:~$ sed '1d' myfile
Hello Bioinformatics!
dgarrimar@5CG6255443:~$ sed 's[l[g[' myfile
Heglo World!
Heglo Bioinformatics!
```

- `awk` it is indeed a programming language on its own! It is really versatile and can be used to perform multiple tasks. As `sed`, it performs a task on **every line** of the input file. Some examples are shown below:

```
dgarrimar@5CG6255443:~$ cat myfile
1 one a,b,c
2 two d,e,f
dgarrimar@5CG6255443:~$ awk '$2=="one"' myfile
1 one a,b,c
dgarrimar@5CG6255443:~$ cat myfile | awk 'BEGIN{OFS="@"}{split($3,array,","); print $1,$2,array[2]}'
1@one@b
2@two@e
dgarrimar@5CG6255443:~$ cat myfile | awk '{if($2=="one"){print "Yeah!"}else{print "Nope!"}}'
Yeah!
Nope!
```

3. Conditionals and loops

if/then statements

- As in any other programming language, conditionals can be built in Bash, using the following syntax:

```
if [ condition ]; then statement; fi
```

```
var="foo"
```

```
if [ "$var" = "foo" ]; then  
    echo "condition1 evaluated as true"  
elif [ -e "bioinfo18" ]; then  
    echo "file \"bioinfo18\" exists (condition true)"  
else  
    echo "condition 1 and 2 evaluated as false"  
fi
```

while and for loops

- Again, analogously to other programming languages, `while` and `for` loops can be built in Bash, using the following syntax:

```
while/for [ condition ]; do statement; done
```

```
counter=0
while [ $counter -lt 10 ]; do
    echo "The counter is $counter"
    let counter=counter+1
done
```

```
ls | while read item; do
    echo "item: $item"
done
```

```
for i in {1..10}; do
    echo item: $i
done
```

```
for i in $( ls ); do
    echo item: $i
done
```

Useful operators

`&&` *and* operator

`||` *or* operator

Comparison operators for strings

`s1 = s2` s1 matches s2

`s1! = s2` s1 does not match s2

`s1 < s2` s1 is less than s2

`s1 > s2` s1 is greater than s2

`-n s1` s1 is not null (length greater than 0)

`-z s1` s1 is null (length 0)

Comparison operators for numbers

`-lt` lower than

`-le` lower or equal than

`-eq` equal than

`-ge` greater or equal than

`-gt` greater than

`-ne` not equal

File operators

`-e` file exists

1. Some tips

Some tips

- Script execution (you already know most of it!)
 - *shebang* (`#!/bin/bash`)
 - `chmod +x <script>`
 - `export PATH=$PATH:/path/to/scripts`
- Use `alias` for some shortcuts
 - e.g. `alias ll="ls -l"`
- `~/.bashrc`

Additional resources

- Bash documentation
 - <https://www.gnu.org/software/bash/manual/bash.pdf>
- Bash *cheatsheets*
 - <https://devhints.io/bash>
 - <https://github.com/LeCoupa/awesome-cheatsheets/blob/master/languages/bash.sh>